

Testing Zephyr's Clock Management Subsystem

Dylan Arturo Velasco Pinilla

restricted

Testing Strategy

Two complementary test configurations:

Static	Dinamic
Runtime features disabled	RUNTIME and SET_RATE enabled
Apply devicetree-defined clock states	Change requests from multiple consumers
Verify transitions and resulting output frequencies	Verify state voting, conflicts, and rank-based selection
Check that each configured path produces the expected rate	Verify locking and requested-rate selection

Test procedure: Establish a known default configuration → perform one clock defined operation → check the API result and affected output rates → restore the setup for the next test.

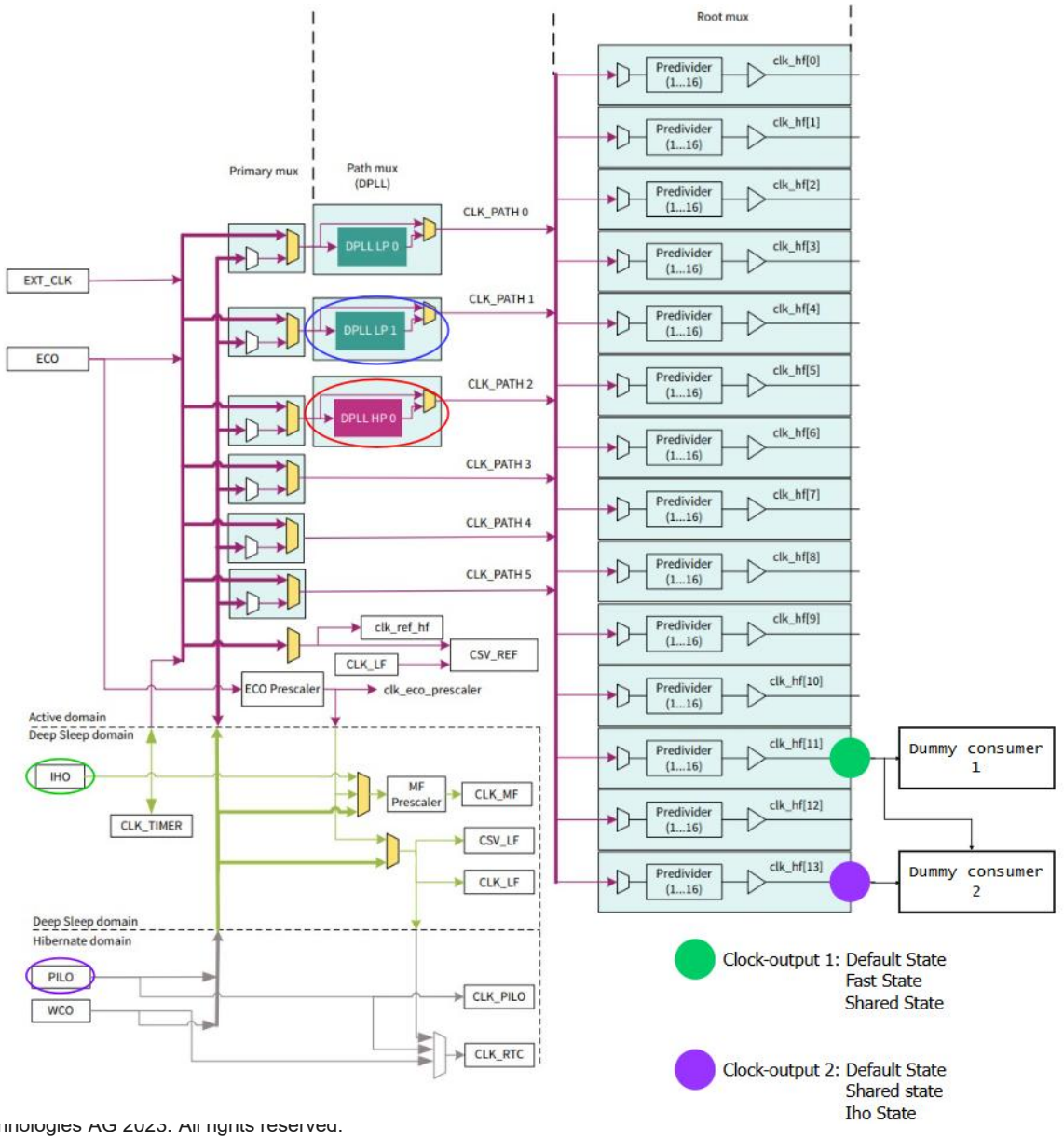
Static tests validate *predefined configurations*; dynamic tests validate *how the subsystem coordinates changes at runtime*.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Tree Under Test

- Two consumers share HF11.
- One also uses HF13.
- Both outputs can depend on common upstream producers.

This topology lets us test direct request conflicts and the effects of rate changes across separate outputs



Copyright © Infineon Technologies AG 2023. All rights reserved.

Clock Output States Under Test

Devicetree-defined paths and expected output frequencies

Clock-Output	STATE	CLOCK – PATH	OUTPUT FREQ
HF11	Default	IHO → PLL LP1 → HF11	133.33 MHz
HF11	Fast	IHO → PLL HP → HF11	450 MHz
HF11	Shared	PILO → PATH MUX 3 → HF11	16.384 kHz
HF13	Default	IHO → PLL HP → HF13	225 MHz
HF13	Shared	PILO → PATH MUX 3 → HF13	32.768 kHz
HF13	IHO	IHO → PATH MUX 3 → HF13	50 MHz

These six states give us known clock paths and expected frequencies. They let us verify state transitions on each output and exercise dependencies through shared upstream producers.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Consumer Clock Requests

A request lists acceptable states for each output

CONSUMER	REQUEST	HF11 Allowed States	HF13 Allowed States
Dummy 1	Default	Default	Default
Dummy 1	Shared	Shared	Shared
Dummy 1	Fast	Fast	Default
Dummy 1	IHO	—	IHO
Dummy 1	All	Default, Shared, Fast	—
Dummy 2	Default	Default	—
Dummy 2	Fast	Fast	—
Dummy 2	Shared	Shared	—
Dummy 2	Locking	Fast, Shared	—

- **Compatible requests:** Dummy 1 selects **All** and Dummy 2 selects **Fast**. Both allow **Fast** on HF11.
- **Conflicting requests:** Consumer 1 selects **Default** and Consumer 2 selects **Fast**. They have no common allowed state for HF11.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Static Clock Configuration Tests

Verify devicetree-defined clock states and output frequencies with runtime management disabled. A setup step requests the default state before each test.

Test	What it verifies
Default state — HF11	Applies the default state and checks HF11's expected frequency. Repeats the rate check while turning the output on and off, confirming the reported rate remains consistent in this setup.
Slow state — HF11 and HF13	Applies the slow request, checks HF11's slow frequency, and confirms HF13 remains at its default frequency. Rechecks both outputs around on/off operations.
Invalid state	Requests a invalid configuration, a PLL with an input frequency too low to lock, and checks that the request returns an error.

These tests check *state application, frequency reporting, on/off operations, and invalid-configuration handling*. They do **not** test runtime voting, reference counting, or notifications.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Runtime State Requests and Shared Producers

Test setup: Before each test, both consumers request their default states. Consumer 1 enables HF11 and HF13; Consumer 2 also enables HF11. This establishes known frequencies and **usage counts**. Cleanup balances the three enable operations.

Test	Scenario	What it verifies
Default clock rates	Both consumers use their default requests.	HF11 reports its expected default rate through **both consumers**, while HF13 reports its own default rate. Consumers sharing HF11 observe the same output frequency.
Conflicting state requests	Consumer 1 requires **Default** on HF11; Consumer 2 requests **Fast**.	With incompatible requests, HF11 **remains at Default** rather than switching to Fast. Once both requests agree on Fast, HF11 reports the Fast frequency. **Known issue:** the incompatible request currently returns success instead of reporting the conflict.
Shared producer	Both consumers move HF11 toward **Shared**. Consumer 1 then changes HF13 from **Shared** to **IHO**; both paths involve Path Mux 3.	HF13 can enter Shared while HF11 remains at Default because Consumer 2 still requires Default. Once Consumer 2 also requests Shared, HF11 changes to Shared. The final request to IHO shows a **cross-output effect**: changing Path Mux 3 for HF13 gives HF13 **50 MHz** and changes HF11 to **25 MHz**.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Runtime Rate, Gating, and Locking Tests

Test	Scenario	What it verifies
Runtime rate request	Consumer 1 requests **100 MHz** on HF13.	The rate-selection path returns **99,999,998 Hz** rather than the requested value.
Clock gating and usage counts	Enable HF11 for both consumers and HF13 for Consumer 1. Disable HF11's clock outputs one at a time, then switch both consumers to *Fast* and re-enable HF11.	Shared producers' usage counts decrease as clock outputs are disabled. Changing a state alone does not increase usage counts; enabling the outputs does.
Clock locking	Consumer 1 locks HF11, then switches to a non-locking request. Consumer 2 takes ownership through a locking request, and Consumer 1 attempts another state change.	Only one consumer can own the lock. The non-locking request releases Consumer 1's lock; Consumer 2's lock causes Consumer 1's subsequent request to be rejected.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Runtime Rate, Gating, and Locking Tests

Test	Scenario	What it verifies
Runtime rate request	Consumer 1 requests **100 MHz** on HF13.	The rate-selection path returns **99,999,998 Hz** rather than the requested value.
Clock gating and usage counts	Enable HF11 for both consumers and HF13 for Consumer 1. Disable HF11's clock outputs one at a time, then switch both consumers to *Fast* and re-enable HF11.	Shared producers' usage counts decrease as clock outputs are disabled. Changing a state alone does not increase usage counts; enabling the outputs does.
Clock locking	Consumer 1 locks HF11, then switches to a non-locking request. Consumer 2 takes ownership through a locking request, and Consumer 1 attempts another state change.	Only one consumer can own the lock. The non-locking request releases Consumer 1's lock; Consumer 2's lock causes Consumer 1's subsequent request to be rejected.

Copyright © Infineon Technologies AG 2023. All rights reserved.

Key Findings from Evaluating Clock Management

1. Recursive validation: state changes require approval from every child → **runtime scales with clock tree size**; three notification phases (query, pre, post).

2. Sequential producer reconfiguration: a failure mid-path leaves the **clock path partially configured** at an undesired frequency.

3. Requested vs. achieved frequency: the best configuration is **hardware-dependent** and can be far from the requested rate.

4. Shared clock paths **force consumers to implement callbacks** in all three phases to survive rate changes.

5. Bug in request_state(): a losing vote (active requests > vote count) **does not return an error** as it should.

6. Driver-delegated robustness: **Relies on robust drivers**; callbacks act as a circuit breaker against harmful reconfigurations.

7. Reliable clock gating: Enables **further power management optimization** by effectively disabling unused clock paths.

8. Powerful abstraction: Manages and reconfigures complex clock trees by abstracting the configuration process.

Copyright © Infineon Technologies AG 2023. All rights reserved.


Infineon